Dexy:
A Simple Stablecoin Design Based on an Algorithmic Central Bank

Alexander Chepurnoy
Ergo Platform
kushti@protonmail.ch

Amitabh Saxena
Ergo Platform

Luca D'Angelo
The Stable Order
ldgaetano@protonmail.com

Abstract

We consider a new algorithmic stablecoin called Dexy, consisting of two assets: the base cryptocurrency, called the basecoin, and the stablecoin. The protocol consits of three components: a central bank, a secondary market in the form of a customized CP-AMM liquidity pool, and a trusted oracle. The price of the stablecoin in the liquidity pool is stabilized, relative to the oracle price, via central bank interventions.

1 Introduction

Due to the inherent volatility of cryptocurrency prices, algorithmic stablecoins naturally arose as their extension. The volatility is resolved by pegging the stablecoin to an asset whose price is considered to be stable over long periods of time, e.g. gold.

Having an asset with a stable value can be useful in many scenarios:

- Securing fundraising: a project can be sure that funds collected during fundraising will have stable value in the mid-to-long term.

We would would like to thank Ile for his inspiring forum posts, Bruno for discussions and the extract to future idea, and 4EYES Labs for the audit of the ErgoScript contracts.

- Doing business with predictable results: a shop can be sure that funds collected from sales will have roughly the same value when the shop is ordering goods from warehouses, otherwise, the shop may go bankrupt if its margin is to low to cover exchange rate fluctuations.
- Shorting: when cryptocurrency prices are high, it is desirable for investors to rebalance their portfolio by increasing exposure to fiat currencies or real-world commodities. However, as fiat currencies and centralized exchanges impose significant risks, it would be better to buy fiat and commodity substitues in the form of stablecoins on decentralized exchanges.
- Lending and other decentralized finance applications: stability of collateral value is critical for many applications built on top of these services.

Centralized stablecoins, such as USDT and USDC, use a trusted party to ensure that the peg is maintained. For algorithmic stablecoins, the peg can be maintained in different ways depending on the implementation, e.g. rebasement of the total supply [4], or via imitating a trusted party that holds reserves and then does market interventions for rebalancing the peg. Imititating the trusted party is usually done by allowing anyone on the blockchain to create over-collateralized financial instruments, such as collateralized debt positions [3], zero-coupon bonds (as was done in the failed Yield protocol), reserve assets [11, 12, 8], or by issuing stabilizing financial instruments in case of a depeg [5].

In this work, we present Dexy, a stablecoin protocol where an algorithmic central bank performs interventions when there is a depeg. The bank stabilizes the stablecoin value in the reference market, a liquidity pool, injecting basecoins from its reserves when the stablecoin price is below the peg. In extreme cases, when bank reserves are depleted and the stablecoin price is still below the peg, the liquidity pool price is restored by extracting stablecoins, which will be injected back again at a later time when the stablecoin price is above the peg by a certain threshold. In some aspects, Dexy could resemble Fei or Gyroscope [7].

The rest of the paper is organized as follows. In Section 2, the Dexy design in general is explained, where important trust assumptions are stated and notation used in other sections is introduced. Section 3 defines the worst-case scenario for the bank reserves, which implies for the stablecoin stability as well. In Section 4, the bank and liquidity pool rules are defined, where their stability is discussed in Section 5. Section 6 describes Dexy implementations: the gold-pegged DexyGold stablecoin and the USD-pegged USE stablecoin.

2 DexyDesign

The Dexyprotocol functions via an algorithmic central bank that holds reserves in basecoins and mints stablecoins. A corresponding liquidity pool exists where stablecoins and basecoins can be traded. The peg price used by the bank is provided by an oracle, which is assumed to be trusted and reliable. Users can interact with both the bank and the liqudity pool, but certain rules and restrictions are applied to control the price stability. Central bank interventions are determined by the ratio of the liquidity pool price to the oracle price, r.

2.1 The Liquidity Pool

The reference market is implemented as an automated market maker (e.g. [10]). For simplicity, a modified constant-product automated market maker (CP-AMM) is used, similar to ErgoDex [1] and UniSwap [6]. This means that the liqudity pools holds a pair of basecoins and stablecoins such that the product of their amounts remains invariant before and after a trade. Users who function as liquidity providers can provide a pair of basecoins and stablecoins to obtain a corresponding amount of LP tokens. These LP tokens can later be deposited to obtain the initial liquidity pair deposit plus accumulated fees from trading. A fee is charged to liqudity providers to prevent excessive liquidity withdrawals.

2.2 The Algorithmic Central Bank

By depositing basecoins into the bank, users can mint stablecoins. The price provided by the bank will always equal the oracle price. A fee is charged so the bank reserves can increase faster. Under certain conditions, the bank can intervene into the liquidity pool to control its price relative to the oracle price.

2.3 Trust Assumptions

In general, for any DeFi protocol involving user funds, it is important for trust assumptions to be explicitly stated so that users can make informed decisions to the best of their ability. The Dexyprotocol has two:

- Oracle: The protocol assumes the target price oracle is reliable. This trust assumption is unavoidable; however, it can be relaxed by using a federation of oracles that produces an average price to be delivered to the central bank, instead of relying on a single entity. In any case, it is important that oracles are rewarded from protocol usage, otherwise, they will be incentivized to adversarially attack the protocol, see Section **??** for details.
- No governance: Unlike most stablecoin protocols, we do not consider governance, as it can be used to adversarially attack the protocol. We suppose that real-world instantiations that do have it should clearly state their governance-related trust assumptions.

2.4 Actions and Dynamics

The following table shows the different actions in the protocol, the r range when the action happens, and the entity who performs the action.

Action	**Range**	**Entity**
FreeMint	$1 - \epsilon_{\text{FREE}} \leq r \leq 1 + \epsilon_{\text{FREE}}$	User
ArbMint	$r > 1 + \epsilon_{\text{FREE}}$	User
InjectBasecoins	$r < 1 - \epsilon_{\text{FREE}}$	Bank
ExtractDexy	$r \leq 1 - \epsilon_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$	Bank
ReleaseDexy	$1 + \epsilon_{\text{RD}} < r < 1 + \epsilon_{\text{FREE}}$	Bank

2.5 Protocol Specification

We now introduce notation that will be used in further sections:

- $s_{\text{BANK}} = (R, S_D)$

3 Worst Case Scenario and Bank Reserves

Possible failure scenarios for such a design, and how to put restrictions and design rules for the system to ensure stability of the stablecoin token price, will now be considered.

The bank intervenes when there is a sudden drop in the basecoin price provided by the oracle and enough time has passed for the markets to have stabilize themselves, with no interventions, but have failed to do so. In our case, the bank is doing interventions based on the stablecoin price in the liquidity pool in comparison with the oracle price. So, the bank's intervention is about injecting its basecoin reserves into the pool, thereby decreasing the basecoin price to match the oracle price.

Consider an example where the basecoins are ERG, and assume that the oracle price crashes from p to s sharply and stands there, and before the crash there were e of ERG and u of stablecoin in the liquidity pool, with pool's price being p. The worst case then is when no liqudity is put into the pool during the period T. With large enough T and large enough R, this assumption is not very realistic: some traders will buy ERGs with their stablecoins anyway, price is usually failing with swings where traders could mint stablecoins thus increasing ERG bank reserves, etc. However, it would be reasonable to consider worst-case scenario, then in the real world Dexy will be even more durable than in theory.

In this case, the bank must intervene after T units of time, as the price differs significantly, and restore the price in the pool, so set it to s. We denote amounts of basecoins and stablecoins in the pool after the intervention as e' and u', respectively. Then:

- $e * u = e' * u'$.
- The bank must inject E_1 basecoins into the pool, thus after the trade there will be $e' = e + E_1$ basecoins in the pool.
- The new basecoin price in the pool will be $\frac{u'}{e'} = s$, thus $u' = s * (e + E_1)$.
- So, isolating for the amount that must be injected, we obtain $E_1 = \sqrt{\frac{e*u}{s}} - e$ basecoins that must be added into the pool from the bank.

So by injecting E_1 basecoins, the bank recovers the price. However, this is not enough, as now there are O stablecoin units which can be injected into the pool from outside. Again, it is not realistic to assume that all the O stablecoin would be injected, as some of them are simply lost, some would be kept to buy cheap basecoins at the bottom (stablecoins are often used as a bet for a basecoin price decline), etc. However, we need to assume the worst-case scenario again. We also unrealistically assume that all the O tokens are being sold in very small batches that do not significantly affecting the pool price, where after each batch, a seller of a new batch is waiting for a bank intervention to happen (so for T units of time), and sells only after the intervention. In this case, all the O tokens are being sold at price close to s, so the bank should have about $E_2 = \frac{O}{s}$ basecoins in reserve to buy the tokens back.

Summing E_1 and E_2, we obtain the basecoin reserves the bank should have to be ready for the worst-case scenario: $E_w = E_1 + E_2 = \sqrt{\frac{e*u}{s}} - e + \frac{O}{s}$.

This scenario is the worst-case for the bank (and only the bank) because it is buying all of the external O $stablecoin$ at the worst possible price s, and also sets the new price by burning its own reserves only.

4 Bank and Pool Rules

Based on needed reserves for worst-case scenario estimation, we can consider minting rules accordingly. Similarly to SigUSD [2] (a Djed instantiation on the Ergo blockchain), we can, for example, target for security in case of a 4x price drop, so to consider $R = \frac{p}{s} = 4$. We can also allow minting of stablecoins while there are enough, so not less than E_w, basecoins in reserves. However, in this case, the stablecoin would be non-mintable most of the time, and only during moments of basecoin price going up significantly will it be possible to mint Dexy. As worst-case scenario is based on unrealistic assumptions, unlikely a realistic protocol can be built on top of it.

Thus, we leave worst-case scenario for UI, so dapps implemented to interact with the Dexy protocol may show, for example, collateralization for the worst-case scenario. Having on-chain data analysis, more precise estimations of reserve quality can be made (by considering stablecoins locked in DeFi protocols, stablecoins likely forgotten in wallets, etc).

Instead, we always allow for cautious minting. That is, we allow minting when oracle price is above pool's price (so the bank provides liquidity for arbitrage when the stablecoin price is above the peg), or we allow to mint a little bit (per some time period) when liquidity pool is in good shape. In details, we have two following minting operations, with minting price being the oracle's price p:

- Arbitrage mint: if price reported by the oracle is higher than in the pool, i.e. $p > \frac{u}{e}$, we allow to print enough stablecoin tokens to bring the price to p. That is, the bank allows to mint up to $\delta_u = \sqrt{p * e * u} - u$ stablecoin by accepting up to $\delta_e = \frac{\delta_u}{p}$ basecoins into reserves.

  To instantiate the rule, we can allow for arbitrage minting if the price p *is more than* $\frac{u}{e}$ *by at least* 1% *for time period* T_{arb} *(e.g. 1 hour), with the bank charging* 0.5% *fee for the operation. After arbitrage mint, it is not possible to do another one within 30 minutes, in order to prevent aggressive liquidity minting via chained transactions, etc.*

- Free mint: we allow to mint up to $\frac{u}{100}$ stablecoins within time period T_{free}.

  To instantiate the rule, we propose to allow for free mint if $0.98 < r < 1.02$*. Minting fee in this case is also* 0.5%*. We propose to set* T_{free} *to* 12 *hours, then bank reserves can grow by* 2% *of LP volume per day when the peg is okay.*

In addition to minting actions, which increase the bank reserves, we define the following action which decreases the bank reserves, but functions as the first stabilization mechanism:

- Intervention: if the price reported by the oracle is lower than the pool price by a significant enough margin, i.e. $r = \frac{p*e}{u}$ is less than or equal to some constant that is set as a protocol parameter, then the bank will provide enough basecoins to restore the pool price to the oracle price.

  To instantiate the rule, we propose to allow the bank to intervene if $r \leq 0.98$ *for time period* $T_{int} = 1\ day$*. During the intervention, the period is reset, so another intervention will only occur after* $T_{int} = 1\ day$ *at least. The bank will provide enough basecoins to get the pool price price to 99.5% of the oracle price max, but making sure to not spend more than 1% of its reserves for that.*

We also state following rules for the liquidity pool (which, otherwise, acts as an ordinary CP-AMM liquidity pool):

- Stopping withdrawals: if r is below some threshold, withdrawals from liquidity providers are stopped (i.e. liquidity providers cannot redeem their LP tokens for basecoins/stablecoins), so only swaps between basecoins and stablecoins are possible.

  To instantiate the rule, we propose to stop withdrawals immediately if $r \leq 0.98$.

- Second stabilization mechanism: what if the bank is out of reserves, but the basecoin price in the liquidity pool is still above the oracle basecoin price? In this case we restore price in the pool by removing liquidity, and there are two possible options here:

  1. Burn: if the bank if empty, and r is below some threshold, it is allowed to burn stablecoins in the pool.

     To instantiate the rule, we propose to burn stablecoin if $r \leq 0.95$ *for a time period* T_{burn}. T_{burn} *must be quite big, e.g.* $1\ week$. *We burn enough stablecoins to return to the state of stopped withdrawals, so to* $r = 0.98$. *After burning, the period is reset, so another burn will be done only after* T_{burn} *amount of time at least.*

  2. Extract for the future: if the bank is empty and r is below some threshold, it is allowed to extract stablecoins from the pool and lock them in a contract, only to be released in the future when the stablecoin price is above the stablecoin oracle price (i.e. the inverse of the basecoin price peg).

     To instantiate the rule, we propose to extract stablecoins if $r \leq 0.95$ *for time period* T_{burn} *(so the same as in burn). To prioritize extracted funds over arbitrage mint, we do not have delay in releasing contract. We burn enough to return to the state of stopped withdrawals, so close to* $r = 0.98$ *(exactly,* $0.97 \leq r \leq 0.98$*). After extraction, the period is reset, so another extraction will be done after* T_{burn} *amount of time at least.*

In addition, we introduce 2% redemption fee for liquidity providers to avoid excessive liquidity hopping.

5 Stability

With second stabilization mechanism (burning or extraction) in place, the price in the reference market will eventually stabilize. However, for liquidity holders, burning is painful, extraction not as much, but still not desirable. Thus, the Dexy protocol is trying to avoid it (unlike other protocols, such as Gyroscope or Fei, where redemption rate fails and falls below 1 immediately when the collateralization ratio falls under 100%) by giving markets time to self-stabilize, and then doing interventions. However, this could mean slower stabilization, in comparison with other protocols, but slower stabilization helps the bank to play with possible adversaries in an environment with information assymetry, since humans always have access to more information than the algorithmic bank, and with more flexible decision making as well.

Dexy is also cautious about minting new stablecoins, which could mean slow growth of the number of stablecoins issued. This could be inconvenient, especially for big players, but the protocol is focused on stability in the first place.

Please note that the liquidity pool is disincentivizing massive bank runs due to its constant-product nature. Actually, massive bank runs are simpler for the bank to resolve, in comparison to the slow drain of the worst-case scenario.

6 Implementations

In this section we consider two concrete implementations of the Dexyprotocol that were made for the Ergo blockchain, a gold-pegged stablecoin named DexyGoldand a USD-pegged stablecoin named USE. The oracle used in both stablecoins is implemented using the same underlying oracle protocol.

6.1 Oracle Pools

6.2 DexyGold

Oracles security is the most important issue for our stablecoin protocol, since this is the only component that does not function autonomously on-chain. Thus, the protocol needs to reward oracles.

Oracle Pools 2.0 framework [9] has support for paying rewards in custom tokens. So we will create GORT (Gold Oracle Reward Token), which will be used to reward oracles and also developers.

X oracles wil get up to $2 \cdot X$ GORTs per epoch (one datapoint update), per Oracle Pool v2 design [9] ($2 \cdot X$ if all the oracles are active). We consider $X = 30$ oracles with one hour long epoch, which means up to 60 GORTs will be released to oracles.

Any transaction in the protocol that requires the oracle datapoint to be provided as a data-input will charge an oracle fee in ERG. This ERG will be sent to a BuyBack contract. The BuyBack contract purchases GORT available in a LP on ErgoDex, which is then distributed to the oracle operators. Thus, the oracle operators receiving GORT tokens can redeem them for the accumulated ERG from fees in the same LP.

In addition to oracles, GORT is used to reward developers. To do that, we consider a flat emission contract, which is releasing the same amount per one hour, so 2 GORT per block during initial period (2 years).

A UI for interacting with the DexyGold protocol is available at `https://crystalpool.cc`.

6.3 USE

6.4 Remarks

The DexyGold protocol has been implemented on the Ergo blockchain. The ErgoScript smart-contracts follow a separation-of-concerns design pattern, thus each one handles its own part of the protocol instead of having one giant contract, this results in a total of 18 ErgoScript contracts required for the implementation. These 18 contracts are orchestrated together with the corresponding off-chain software for the different protocol interactions and trackers necessary to check the status of the oracle and pool peg, along with the relevant delay periods. The smart-contracts, off-chain code, simulations, and tests can be found in the following repository `https://github.com/kushti/dexy-stable`.

References

[1] ErgoDEX. `https://www.ergodex.io`.

[2] SigUSD. `https://sigmausd.io`.

[3] The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. `https://makerdao.com/en/whitepaper/`.

[4] Ampleforth protocol. [Online], 2018. `https://www.ampleforth.org/`.

[5] XTN - Neutrino Index Token. [Online], 2023. `https://github.com/waves-exchange/neutrino-contract/blob/master/docs/Neutrino%202.0.%20XTN%20Litepaper.pdf`.

[6] Hayden Adams, Noah Zinsmeister, River Keefer, Dan Robinson, and Moody Salem. Uniswap v3 core, 2021. `https://app.uniswap.org/whitepaper-v3.pdf`.

[7] Andrew Breslin. Gyroscope: A Self-Stabilizing, "All-Weather" Reserve-Backed Stablecoin. [Online], 2022. `https://consensys.io/blog/gyroscope-a-self-stabilizing-all-weather-reserve-backed-stablecoin`.

[8] Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025.

[9] scalahub. Eip-0023 oracle pool 2.0. `https://github.com/ergoplatform/eips/pull/41`.

[10] Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Computing Surveys*, 2021.

[11] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. `https://eprint.iacr.org/2021/1069`.

[12] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. `https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program`.

Received \jreceived